Real-world (build and search a real knowledge base) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `knowledge_base_search_practice.py`: writing every document in the small knowledge base yourself means you're your own ground truth — you already know exactly which document should answer which question. So when a search comes back with the wrong chunk, you know it's wrong immediately, instead of having to guess whether the retrieval or the answer is at fault. **If not:** the Build Task's knowledge base would be the first place you ever had to debug a retrieval bug without knowing what the right answer was supposed to be.

Every example below assumes the same 6 sample documents (`docs/*.txt`):

```
# knowledge_base_search_practice.py
DOCS: dict[str, str] = {
    "French press basics.txt": (
        "A French press steeps coarse coffee grounds directly in hot "
        "water, then a mesh plunger separates the grounds. Steep for "
        "about 4 minutes before pressing. It produces a fuller-bodied, "
        "slightly gritty cup compared to filtered methods."
    ),
    "Pour-over basics.txt": (
        "Pour-over brewing means pouring hot water over grounds in a "
        "paper filter, in slow circular motions, letting it drip through "
        "into a cup or carafe below. It produces a clean, light-bodied "
        "cup because the paper filter traps most oils and fine particles."
    ),
    "Grind size guide.txt": (
        "Grind size should match your brew method. Use a coarse grind "
        "for French press, a medium grind for pour-over and drip "
        "machines, and a fine grind for espresso. Too fine a grind for "
        "the method causes over-extraction and bitterness."
    ),
    "Water temperature.txt": (
        "Water for brewing coffee should be between 195 and 205 degrees "
        "Fahrenheit, just off a full boil. Water that's too cool "
        "under-extracts and tastes sour or weak. Water that's too hot "
        "can scald the grounds and taste bitter."
    ),
    "Coffee-to-water ratio.txt": (
```

```
        "A common starting ratio is 1 gram of coffee to 16 grams of "
        "water, sometimes written as 1:16. For a stronger cup, move "
        "toward 1:14. For a milder cup, move toward 1:18. Adjust from "
        "there based on taste."
    ),
    "Storing coffee beans.txt": (
        "Store coffee beans in an airtight container, away from light "
        "and heat. Whole beans stay fresh for a few weeks after "
        "roasting; ground coffee stales much faster, within days. Don't "
        "store beans in the freezer for everyday use, since condensation "
        "on thawing hurts flavor."
    ),
}
```

Solution

```
# knowledge_base_search_practice.py
from pathlib import Path
import chromadb
from openai import OpenAI

openai_client = OpenAI()

def write_sample_docs(doc_folder: str, docs: dict[str, str]) -> None:
    folder_path = Path(doc_folder)
    folder_path.mkdir(exist_ok=True)
    for filename, text in docs.items():
        (folder_path / filename).write_text(text)

def embed(text: str) -> list[float]:
    response = openai_client.embeddings.create(
        model="text-embedding-3-small", input=text,
    )
    return response.data[0].embedding

def build_vector_store(doc_folder: str):
    chroma_client = chromadb.Client()
    collection = chroma_client.create_collection(name="knowledge_base")
    for file_path in Path(doc_folder).iterdir():
        text = file_path.read_text()
        embedding = embed(text)
        collection.add(
            documents=[text], embeddings=[embedding], ids=[file_path.name],
        )
    return collection
```

```
def ask(collection, question: str) -> None:
    question_embedding = embed(question)
    results = collection.query(
        query_embeddings=[question_embedding], n_results=3
    )
    print(f"Q: {question}")
    print(results["ids"][0])
    print(results["documents"][0])
    print()

# Set up: write a few sample coffee docs, then build the vector store.
write_sample_docs("docs", DOCS)
collection = build_vector_store("docs")

# Ask questions — each prints the 3 closest documents.
questions = [
    "What grind size should I use for a French press?",
    "How hot should the water be when brewing?",
    "How long does ground coffee stay fresh?",
]
for question in questions:
    ask(collection, question)
```